

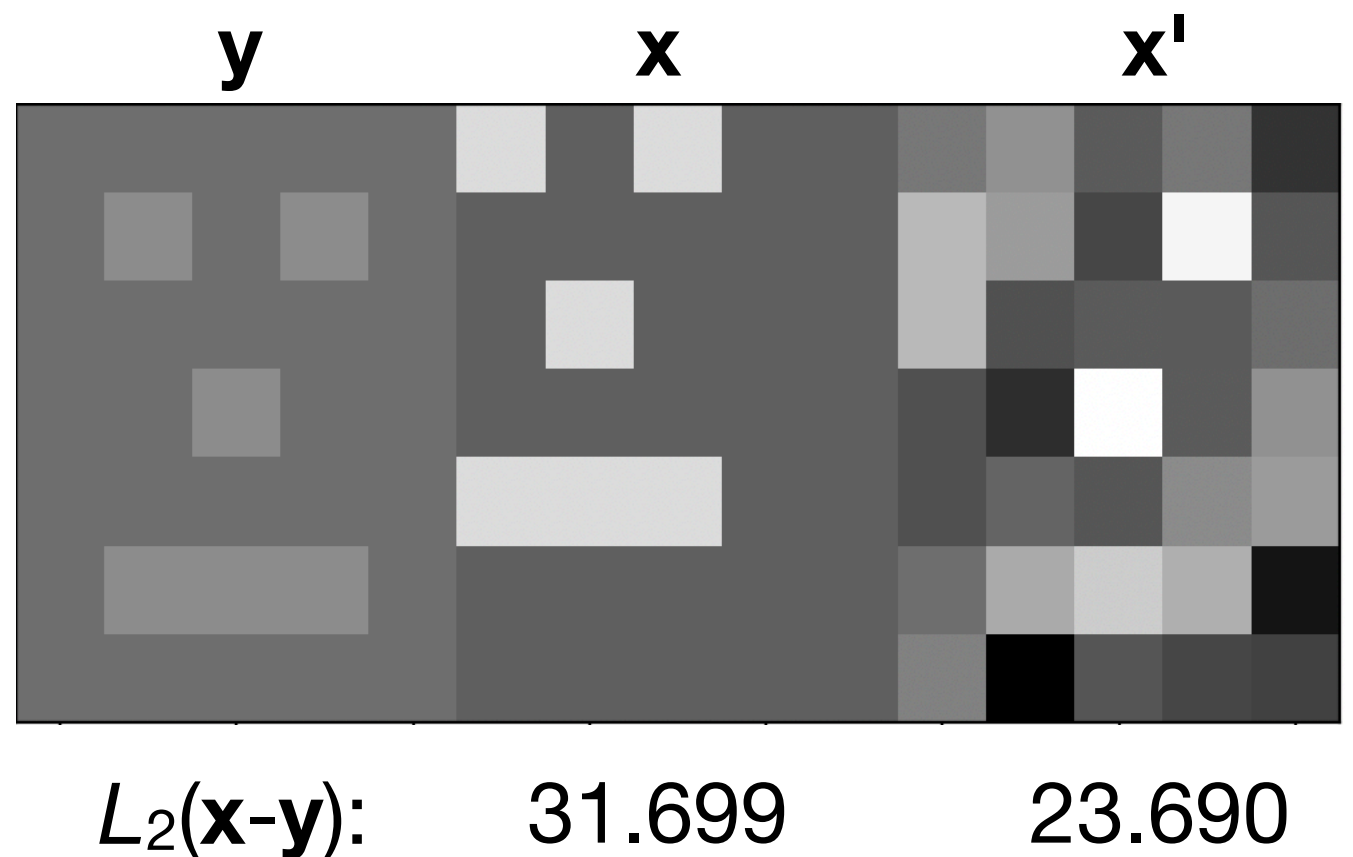
CS/DS 552: Class 10

Jacob Whitehill

Accuracy metrics in image generation

L_2 reconstruction loss

- Suppose you wanted to generate an image $\mathbf{x}$ that looks similar to the specified $\mathbf{y}$.
- Which one looks more similar to $\mathbf{y}$: $\mathbf{x}$ or $\mathbf{x}'$?
- What does the L_2 distance say?
- $\mathbf{x}$ is intuitively much better but has a worse L_2 score than $\mathbf{x}'$
=> wrong metric!

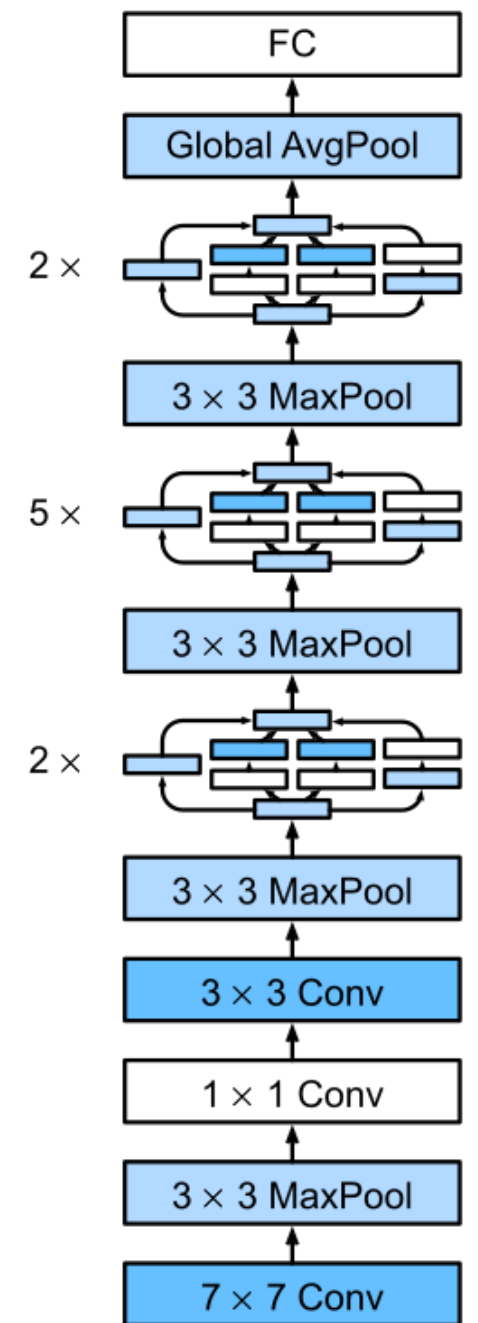


Accuracy metrics in image generation

- What matters more in generated image quality is whether the objects are semantically meaningful and diverse.
- How can we assess the semantic content of an image?
 - High-quality CNNs to extract image content (e.g., for image recognition & object detection) have existed for 15+ years.

Fréchet Inception Distance (FID)

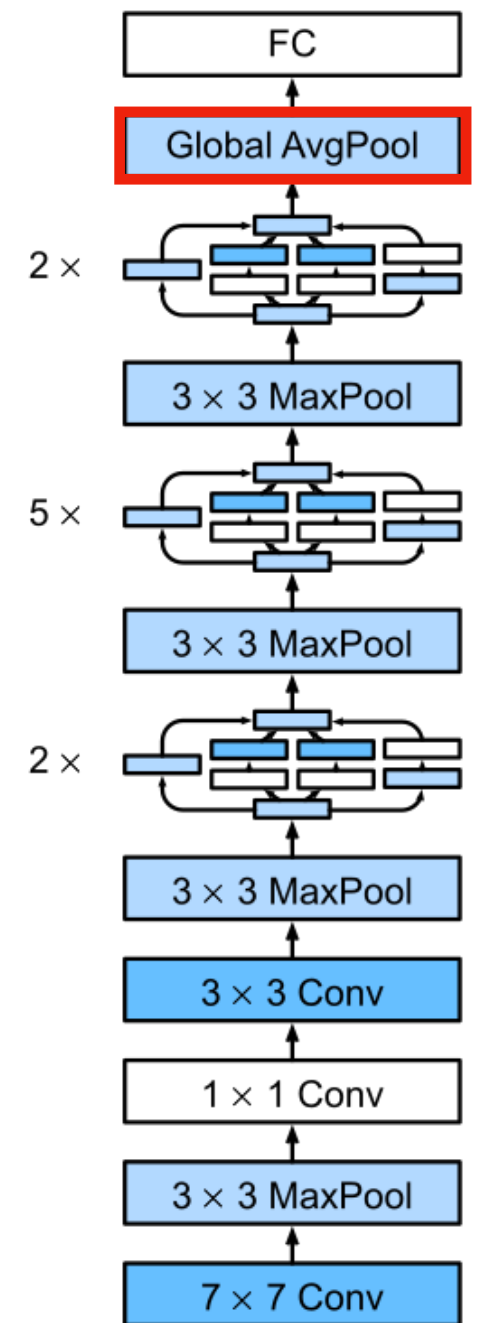
- With FID, the quality of a generated image dataset is measured by comparing *extracted image features* using a NN (Inception) to those of a *reference dataset* (e.g., ImageNet).
- Inception is a CNN (first built in 2014) with powerful feature extraction abilities.



[Inception CNN](#)

Fréchet Inception Distance (FID)

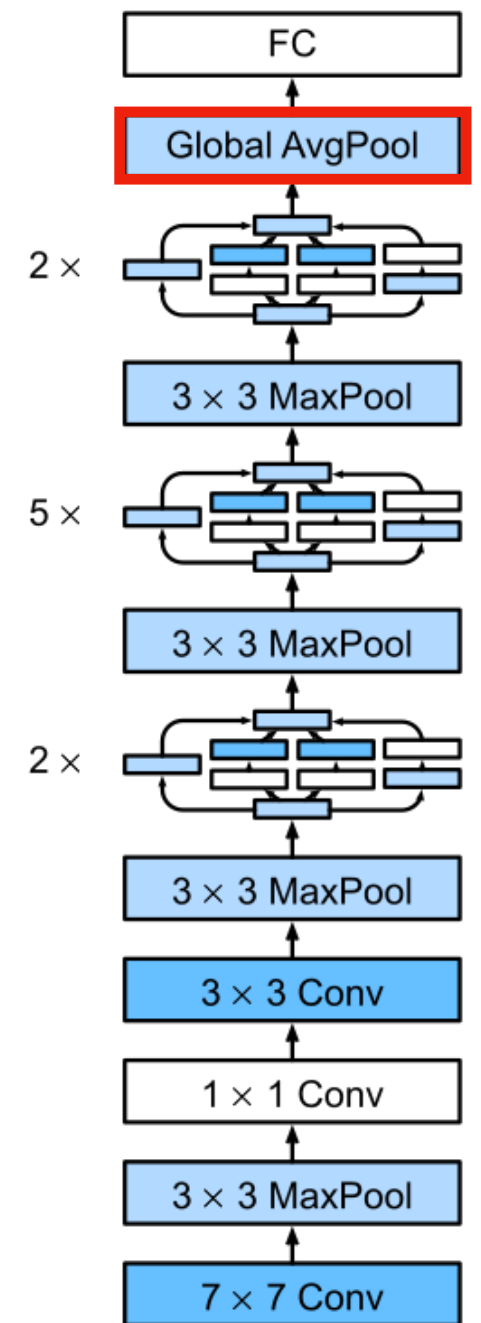
- Procedure:
 1. Extract feature vector $\mathbf{f}^{(i)}$ from Inception's final pooling layer for each image $i=1, \dots, n$.



[Inception CNN](#)

Fréchet Inception Distance (FID)

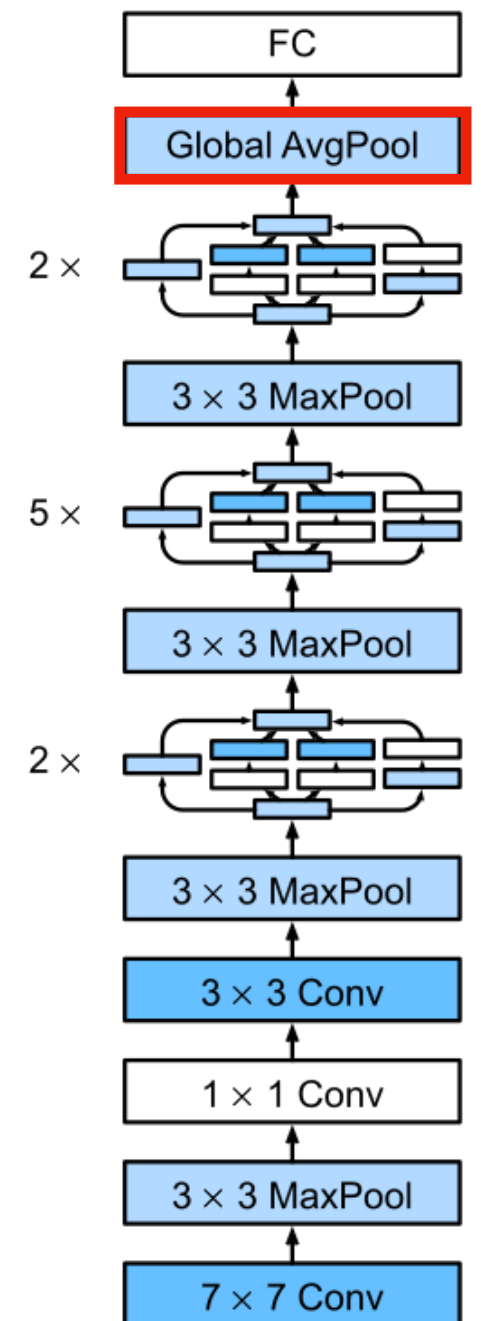
- Procedure:
 1. Extract feature vector $\mathbf{f}^{(i)}$ from Inception's final pooling layer for each image $i=1, \dots, n$.
 2. Estimate a Gaussian distribution $p(\mathbf{f})$ over $\{\mathbf{f}^{(i)}\}_{i=1}^n$ with mean μ_p and covariance Σ_p .



[Inception CNN](#)

Fréchet Inception Distance (FID)

- Procedure:
 1. Extract feature vector $\mathbf{f}^{(i)}$ from Inception's final pooling layer for each image $i=1, \dots, n$.
 2. Estimate a Gaussian distribution $p(\mathbf{f})$ over $\{\mathbf{f}^{(i)}\}_{i=1}^n$ with mean μ_p and covariance Σ_p .
 3. Do the same for the reference dataset to obtain $q(\mathbf{f})$ with mean μ_q and covariance Σ_q .



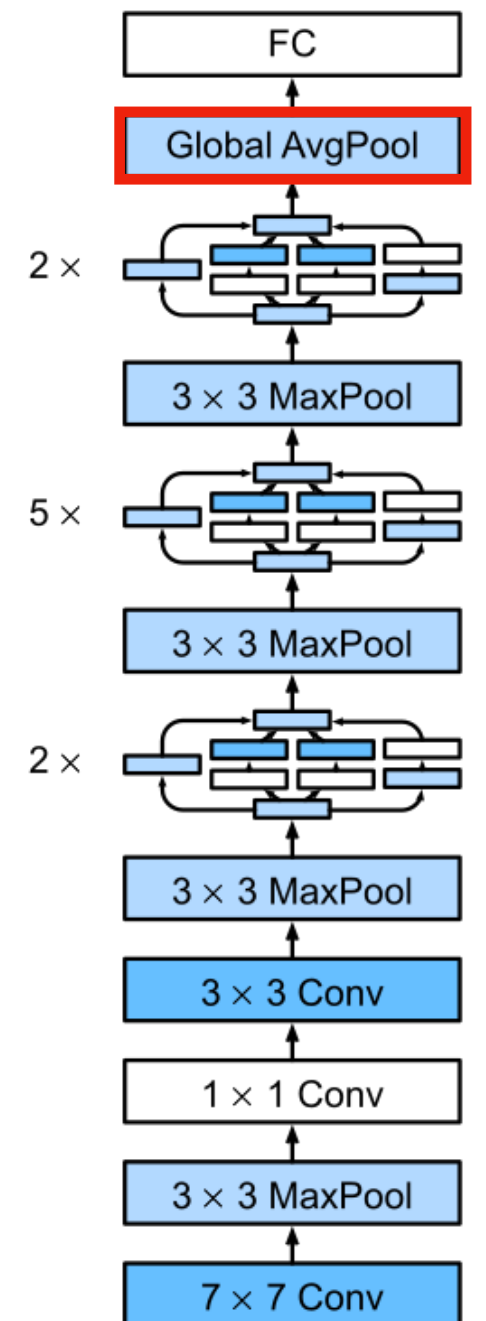
[Inception CNN](#)

Fréchet Inception Distance (FID)

- Procedure:

1. Extract feature vector $\mathbf{f}^{(i)}$ from Inception's final pooling layer for each image $i=1, \dots, n$.
2. Estimate a Gaussian distribution $p(\mathbf{f})$ over $\{\mathbf{f}^{(i)}\}_{i=1}^n$ with mean μ_p and covariance Σ_p .
3. Do the same for the reference dataset to obtain $q(\mathbf{f})$ with mean μ_q and covariance Σ_q .
4. Compute the Fréchet distance d (another similarity metric between probability distributions) between $p(\mathbf{f})$ and $q(\mathbf{f})$:

$$d(p, q) = \|\mu_p - \mu_q\|_2^2 + \text{tr} \left(\Sigma_p + \Sigma_q - 2(\Sigma_p \Sigma_q)^{\frac{1}{2}} \right)$$



[Inception CNN](#)

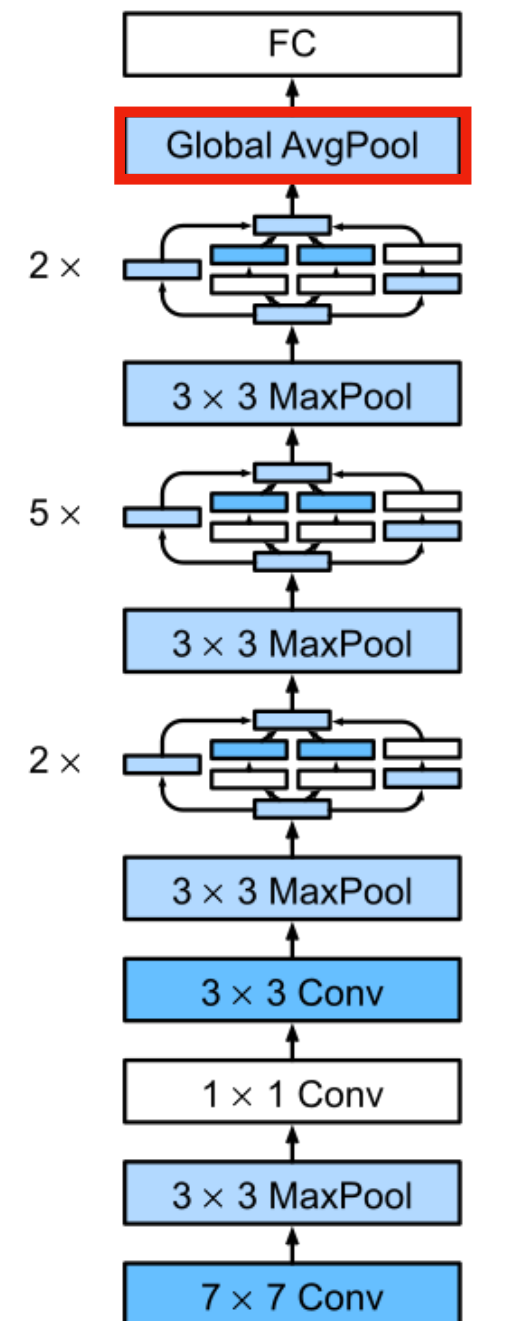
Fréchet Inception Distance (FID)

- Procedure:

1. Extract feature vector $\mathbf{f}^{(i)}$ from Inception's final pooling layer for each image $i=1, \dots, n$.
2. Estimate a Gaussian distribution $p(\mathbf{f})$ over $\{\mathbf{f}^{(i)}\}_{i=1}^n$ with mean μ_p and covariance Σ_p .
3. Do the same for the reference dataset to obtain $q(\mathbf{f})$ with mean μ_q and covariance Σ_q .
4. Compute the Fréchet distance d (another similarity metric between probability distributions) between $p(\mathbf{f})$ and $q(\mathbf{f})$:

$$d(p, q) = \|\mu_p - \mu_q\|_2^2 + \text{tr} \left(\Sigma_p + \Sigma_q - 2(\Sigma_p \Sigma_q)^{\frac{1}{2}} \right)$$

[Show FID examples](#)

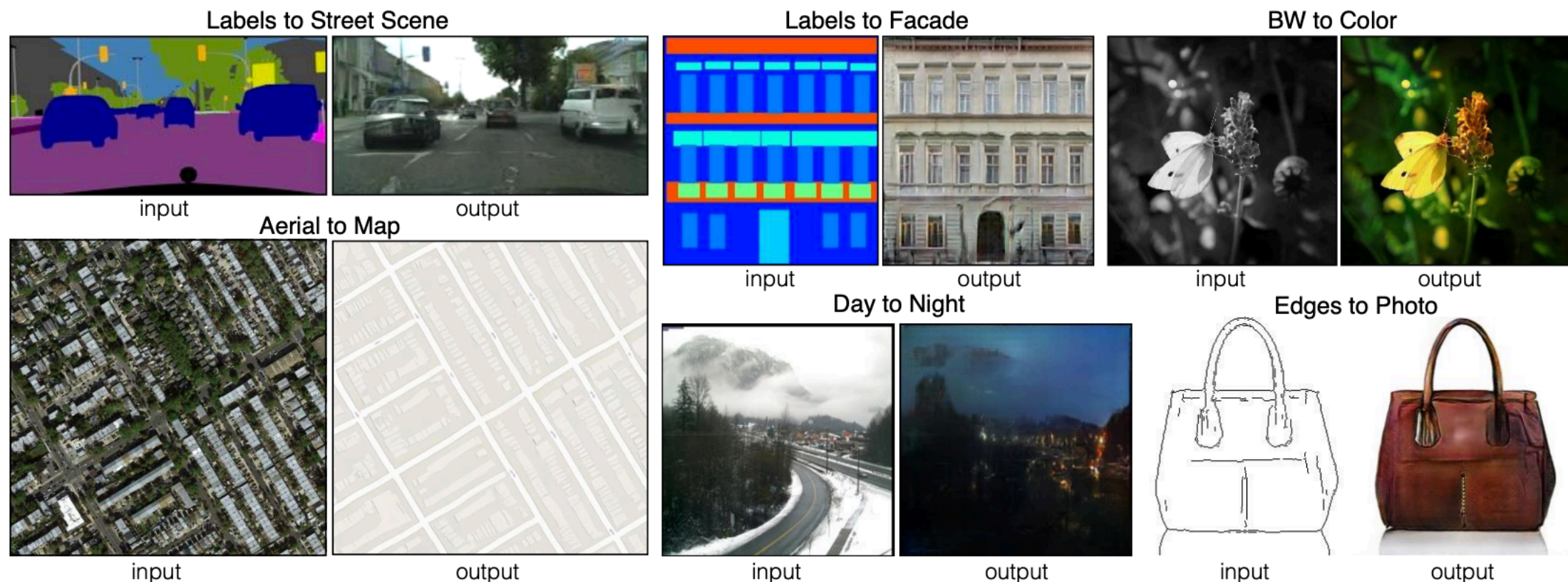


[Inception CNN](#)

**image->image
translation**

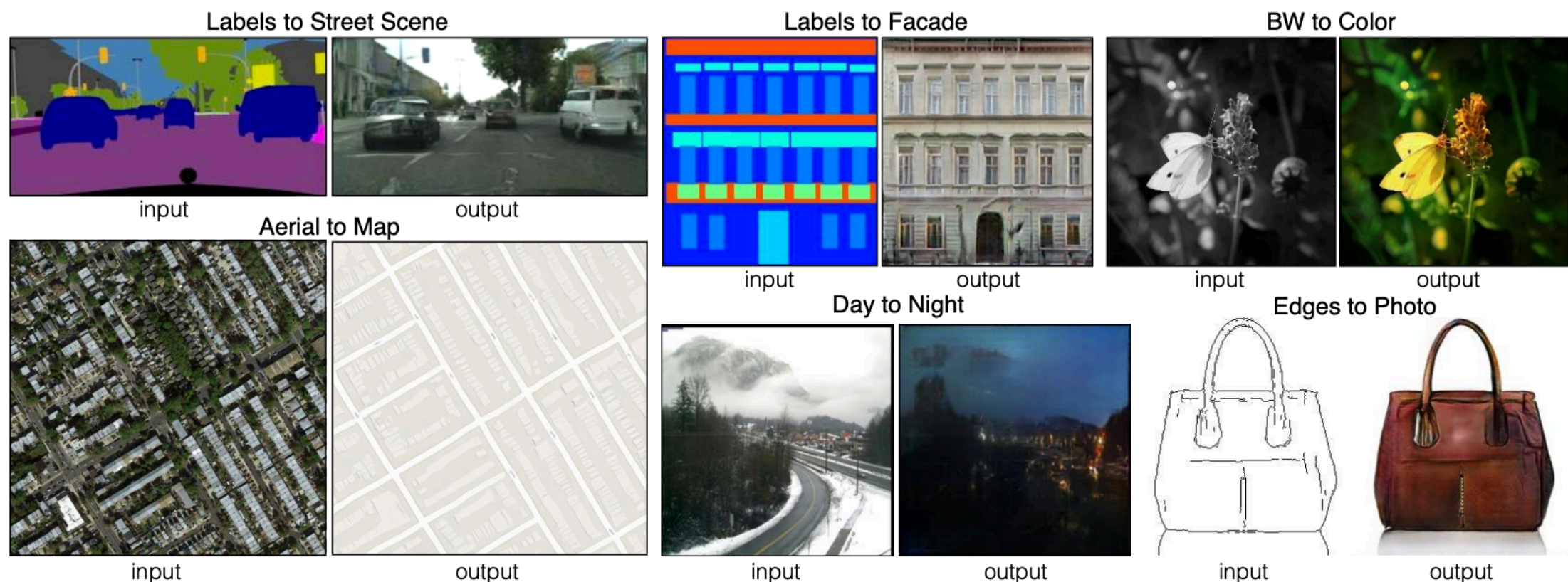
pix2pix (Isola et al. 2018)

- Goal: map an image y into a “stylized” version x of same size.
- Approach: train a **U-Net** as a **c-GAN** to distinguish between real and fake (x,y) pairs.
- Even though this is a “predictive” task, we can still pose it as a conditional generative task.
- Example tasks:



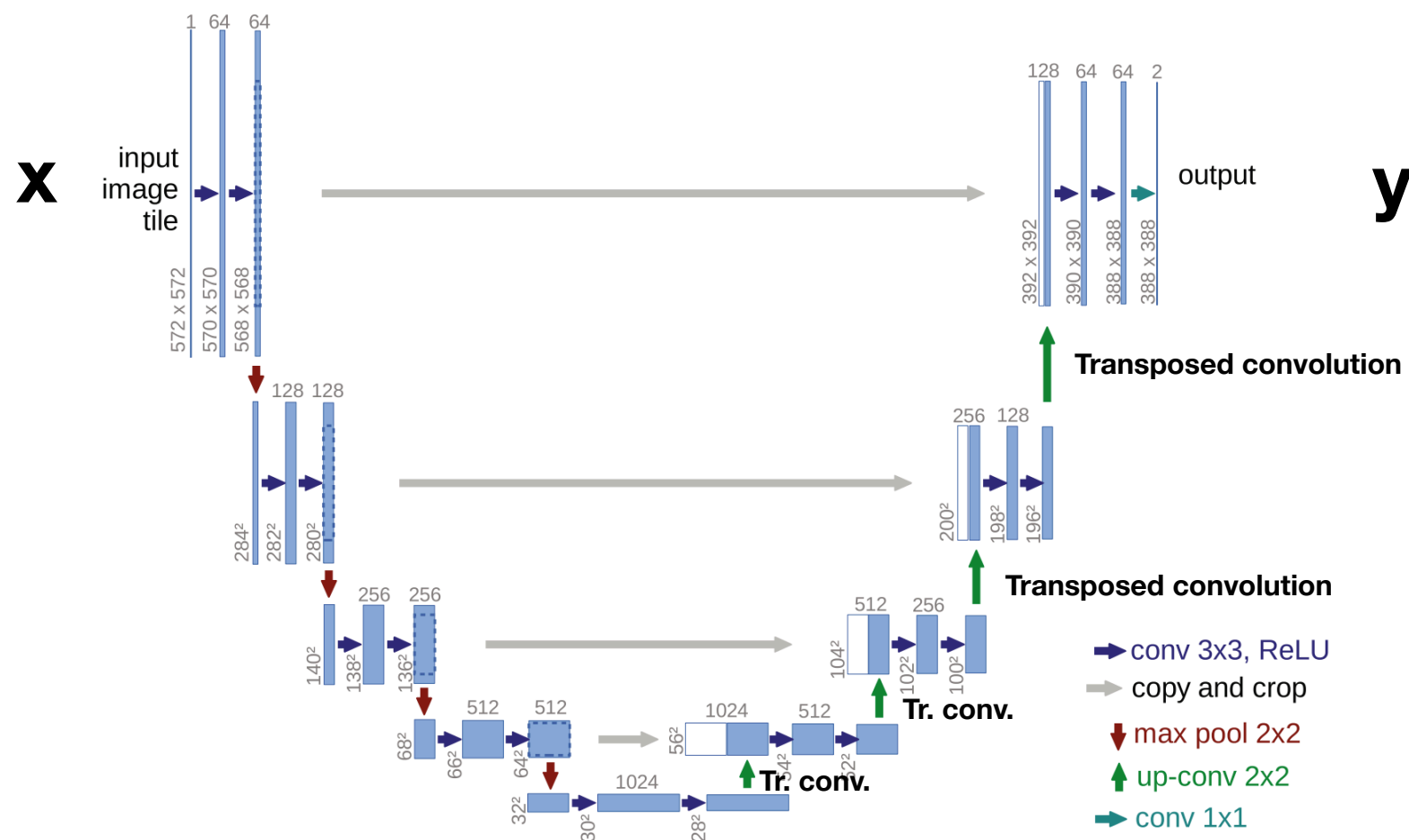
pix2pix (Isola et al. 2018)

- Training a generative model to just minimize L_2 loss will not work well.
- Instead, a c-GAN provides a custom loss function — learnt by the discriminator — that can give better quality.



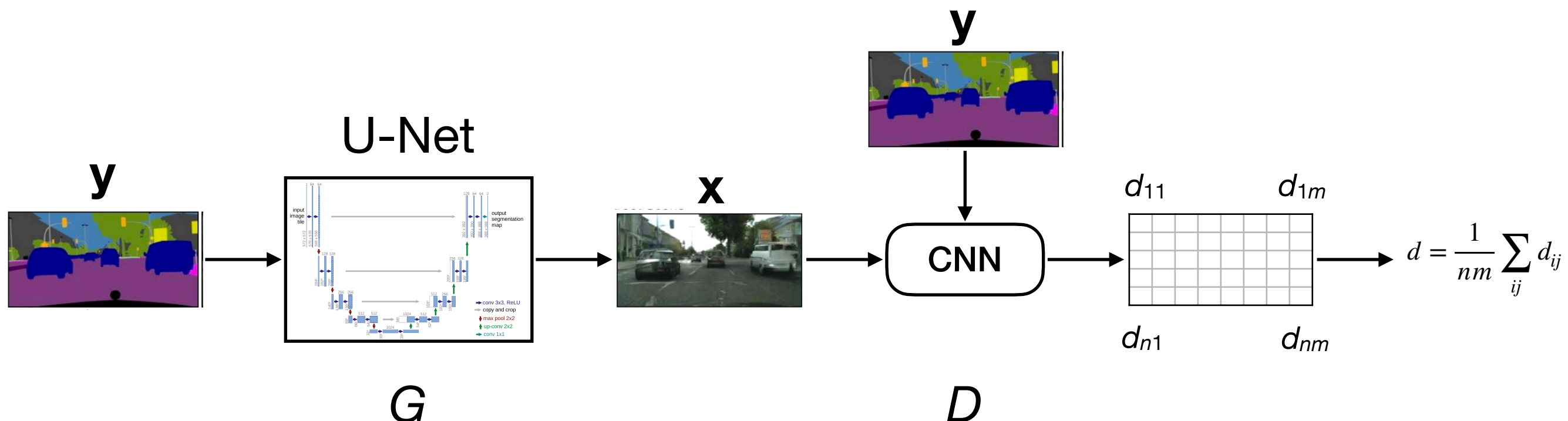
U-Net

- A **U-Net** (Ronneberger et al. 2015) maps an image $\mathbf{x}$ into an image $\mathbf{y}$ of the same size.
- The architecture captures both *high-level semantics* via convolution & down-sampling, as well as *low-level details* with skip connections.



pix2pix: architecture

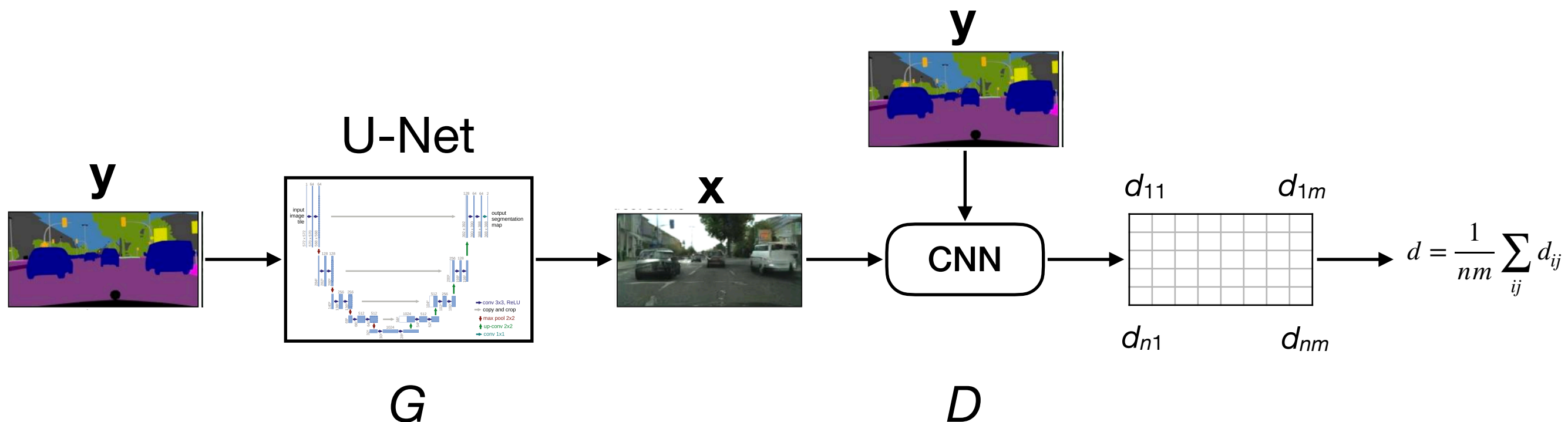
- G is a U-Net and takes an input image y as context.
- In contrast to most c-GANs, pix2pix's G uses **no noise vector z** , i.e., it is deterministic.
- D takes image x (either generated or real) and context y to produce **patch-wise estimates** of fake/real, which are summed to produce a final discrimination score d .



pix2pix: loss

- G is trained both to maximally confuse D but also to minimize the L_1 distance between $\mathbf{x}$ and ground-truth image $\mathbf{t}$:

$$f(\mathbf{y}, \mathbf{t}; \theta_G) = f_{\text{cGAN}}(G(\mathbf{y}), \mathbf{y}, D) + \|G(\mathbf{y}) - \mathbf{t}\|_1$$

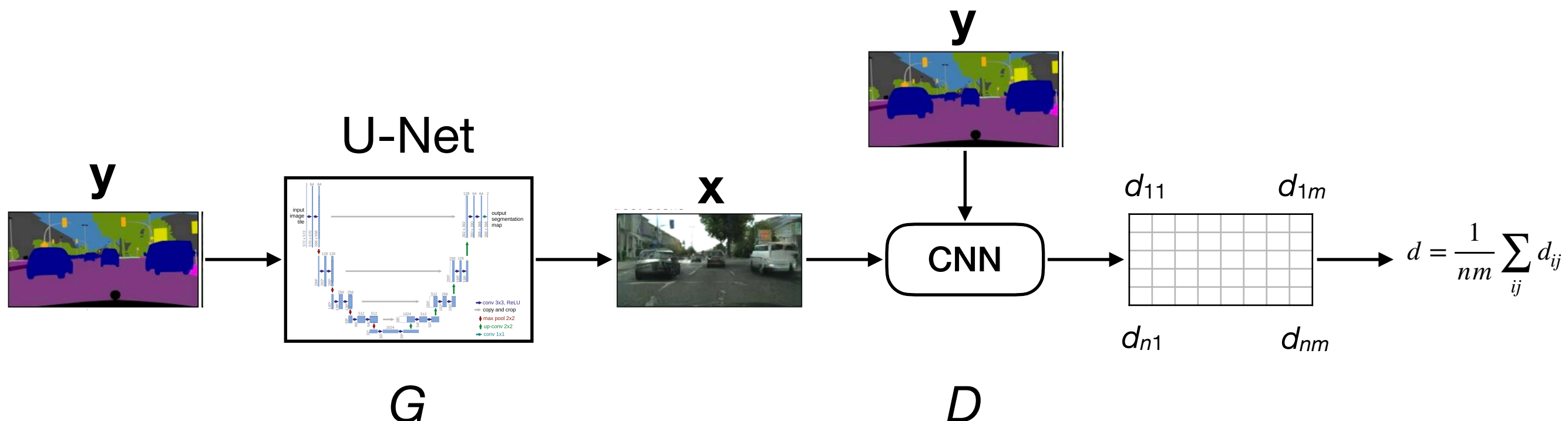


pix2pix: loss

- G is trained both to maximally confuse D but also to minimize the L_1 distance between $\mathbf{x}$ and ground-truth image $\mathbf{t}$:

$$f(\mathbf{y}, \mathbf{t}; \theta_G) = f_{\text{cGAN}}(G(\mathbf{y}), \mathbf{y}, D) + \|G(\mathbf{y}) - \mathbf{t}\|_1$$

Content loss: keep the translation pixel-wise similar to the target

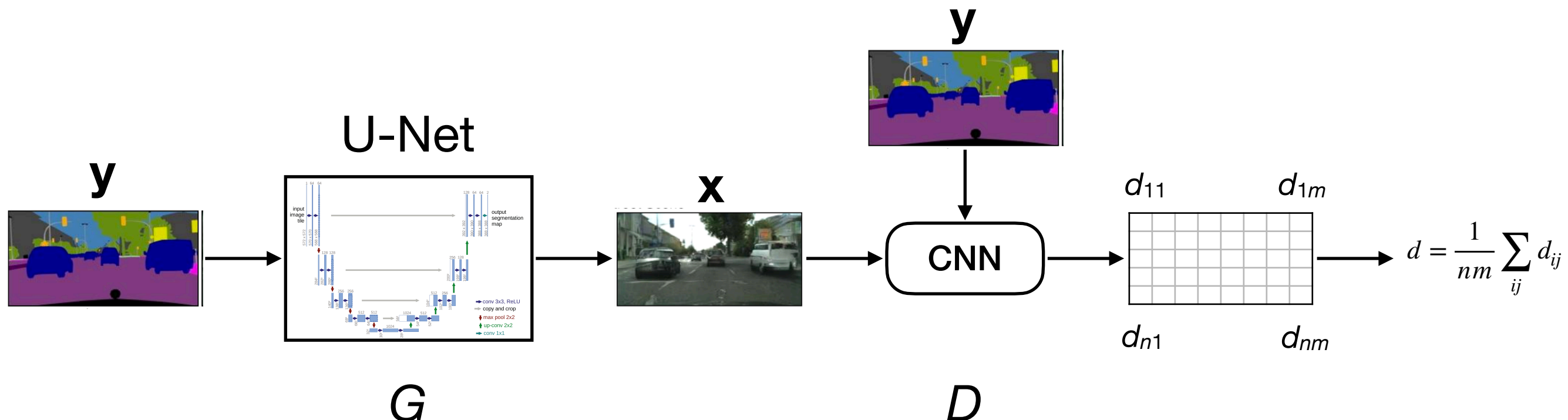


pix2pix: loss

- G is trained both to maximally confuse D but also to minimize the L_1 distance between $\mathbf{x}$ and ground-truth image $\mathbf{t}$:

$$f(\mathbf{y}, \mathbf{t}; \theta_G) = f_{\text{cGAN}}(G(\mathbf{y}), \mathbf{y}, D) + \|G(\mathbf{y}) - \mathbf{t}\|_1$$

cGAN/adversarial loss: ensure the generated image can fool the discriminator



Both losses are needed

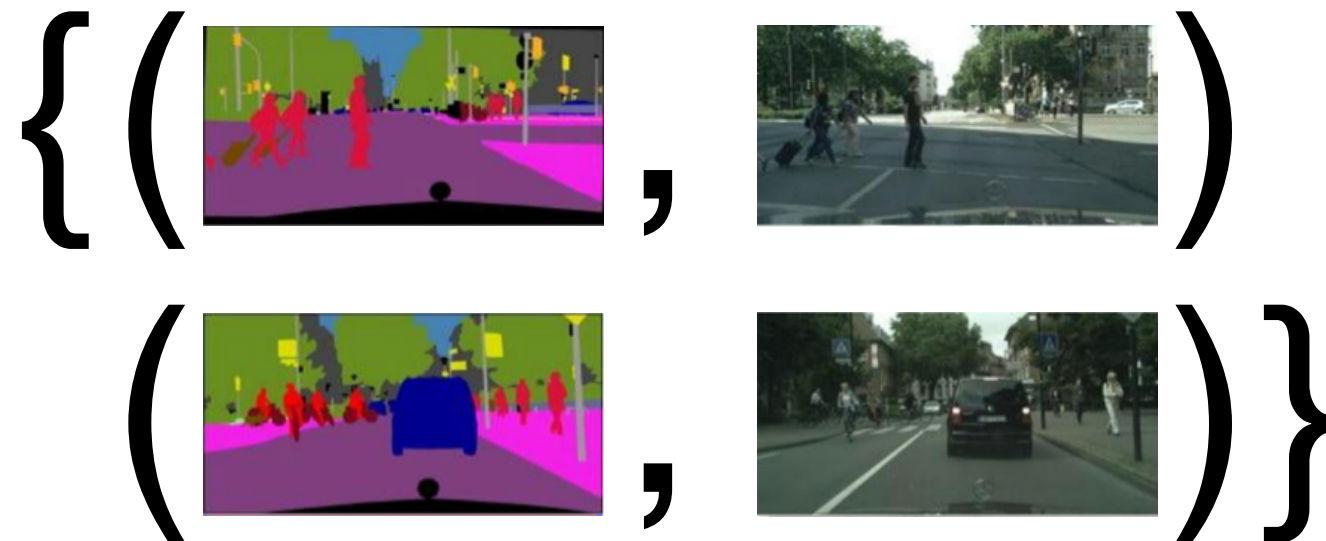
- L_1 loss alone produces blurry images but they are semantically consistent with the input.
- cGAN loss alone produces sharp images but they may contain extraneous objects.
- $L_1 + \text{cGAN}$ gives best results for image generation.



CycleGAN

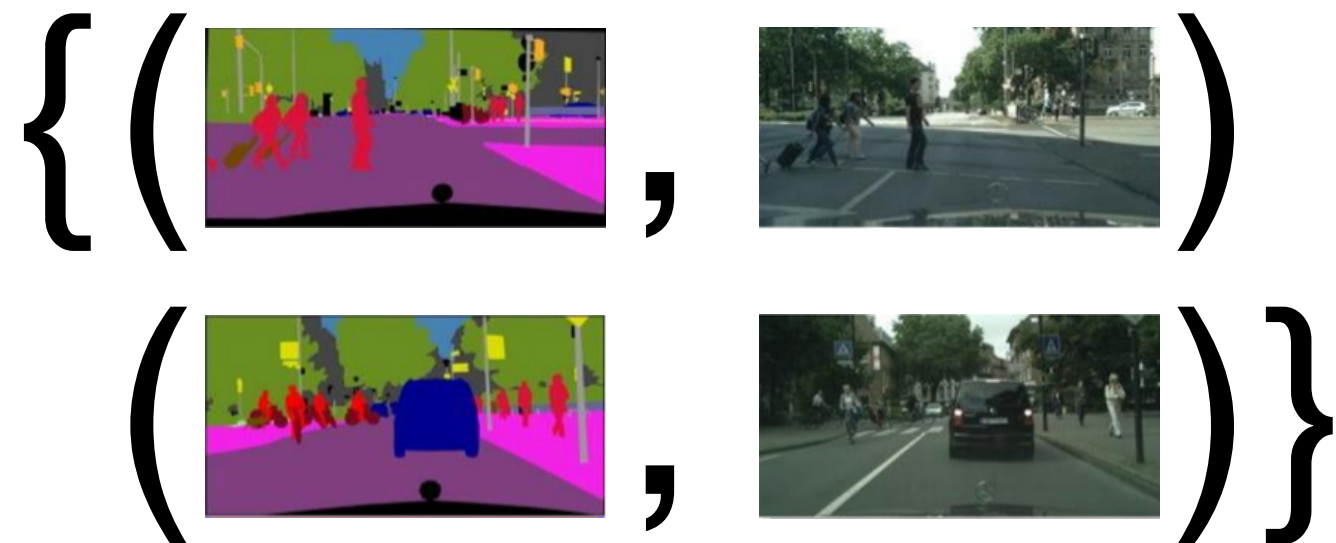
CycleGAN

- pix2pix required training data of (before, after) image *pairs*, e.g.:

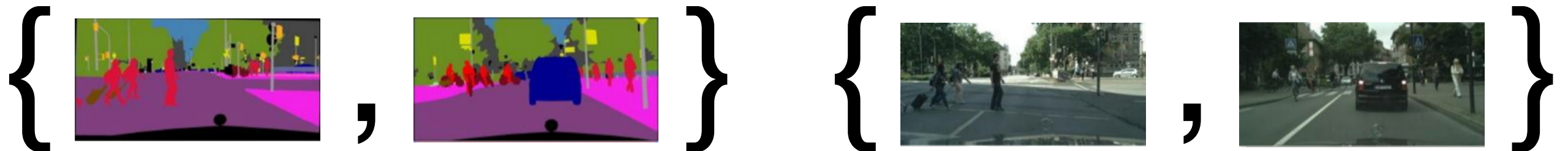


CycleGAN

- pix2pix required training data of (before, after) image *pairs*, e.g.:



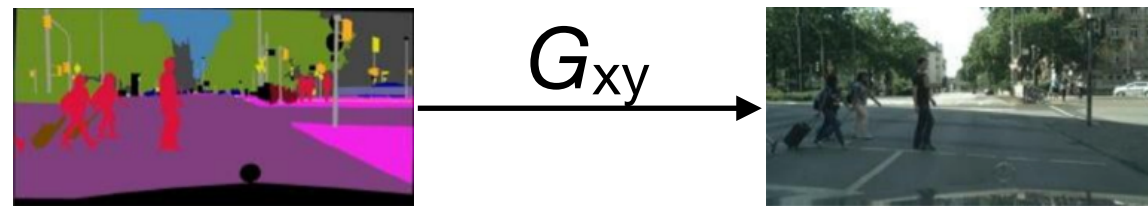
- But what if we only have two sets of images without their pairwise correspondences, e.g.?



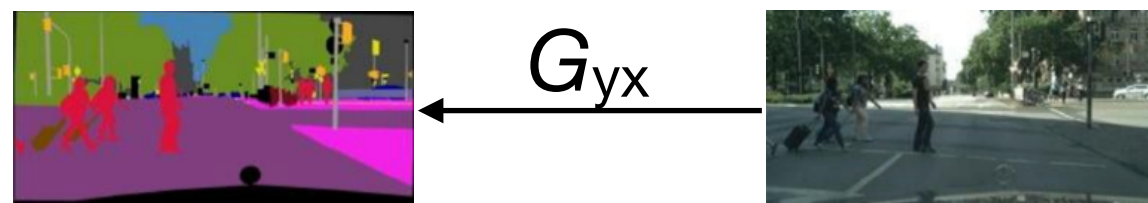
CycleGAN

- CycleGAN performs image->image translation *without* the need for (before, after) pairs.
- It relies on 2 different conditional generators G_{xy} and G_{yx} :

- G_{xy} : translate from domain **x** to domain **y**:



- G_{yx} : translate from domain **y** to domain **x**:



CycleGAN

- CycleGAN performs image->image translation *without* the need for (before, after) pairs.
- It also relies on 2 different discriminators D_x and D_y :

- D_x : distinguish a real $\mathbf{x}$ from a fake (generated) $\mathbf{x}$:



real



fake

- D_y : distinguish a real $\mathbf{y}$ from a fake (generated) $\mathbf{y}$:



real



fake

CycleGAN

- Each generator (G_{xy} and G_{yx}) is trained to optimize the sum of 2 different loss functions:
- **Cycle-consistency loss:** translating "there and back again" should approximate an identity map. Hence, minimize:

$$\|G_{yx}(G_{xy}(\mathbf{x})) - \mathbf{x}\|_1 \text{ and } \|G_{xy}(G_{yx}(\mathbf{y})) - \mathbf{y}\|_1$$

CycleGAN

- Each generator (G_{xy} and G_{yx}) is trained to optimize the sum of 2 different loss functions:
- **Cycle-consistency loss:** translating "there and back again" should approximate an identity map. Hence, minimize:

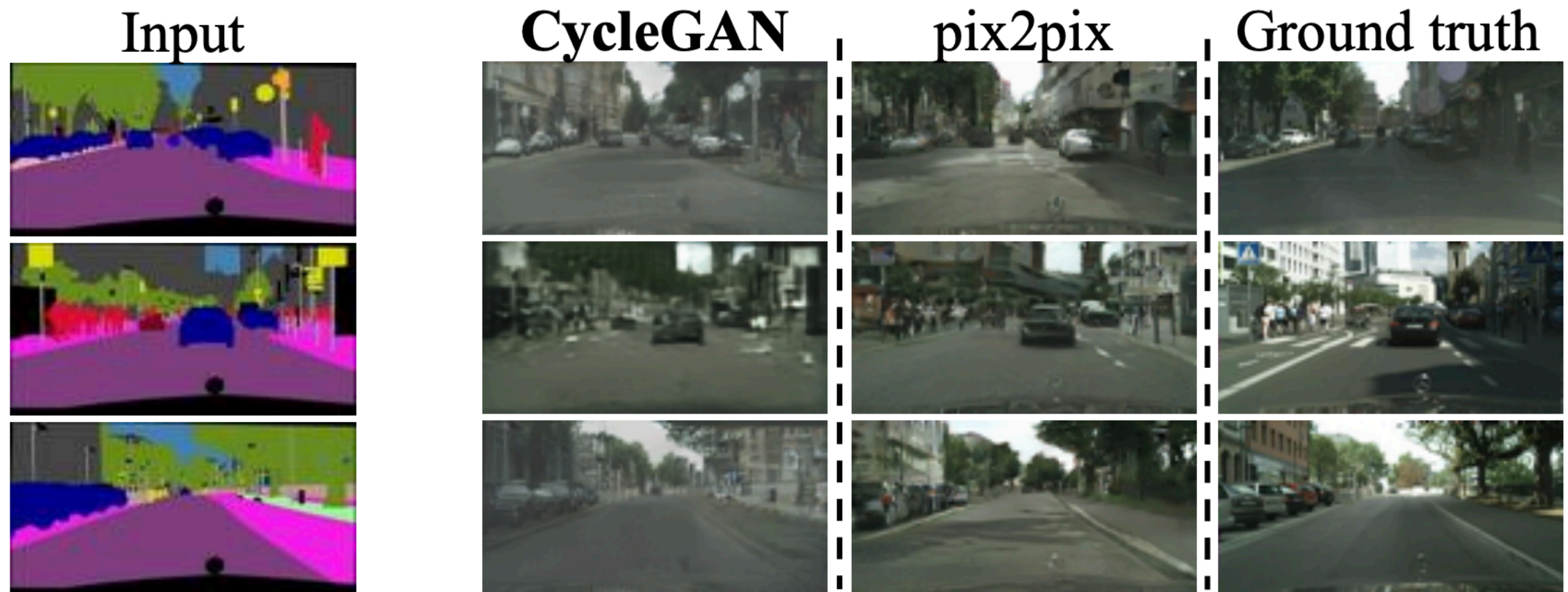
$$\|G_{yx}(G_{xy}(\mathbf{x})) - \mathbf{x}\|_1 \text{ and } \|G_{xy}(G_{yx}(\mathbf{y})) - \mathbf{y}\|_1$$

- **Adversarial loss:** ensure each generated image can fool the discriminator. Hence, minimize:

$$f_{\text{adv}}(\mathbf{x}, D_x) \text{ and } f_{\text{adv}}(\mathbf{y}, D_y)$$

Results:

pix2pix v. CycleGAN



Adversarial loss

- We can conceptualize the **adversarial loss** provided by a discriminator D as a **custom loss function** to help G make realistic images.
- In contrast to L_1 and L_2 losses, it **cares less about trivial pixel differences** and **cares more about semantic differences**.
- The adversarial loss is **trained together with G** and is **specific to a particular generation task**.

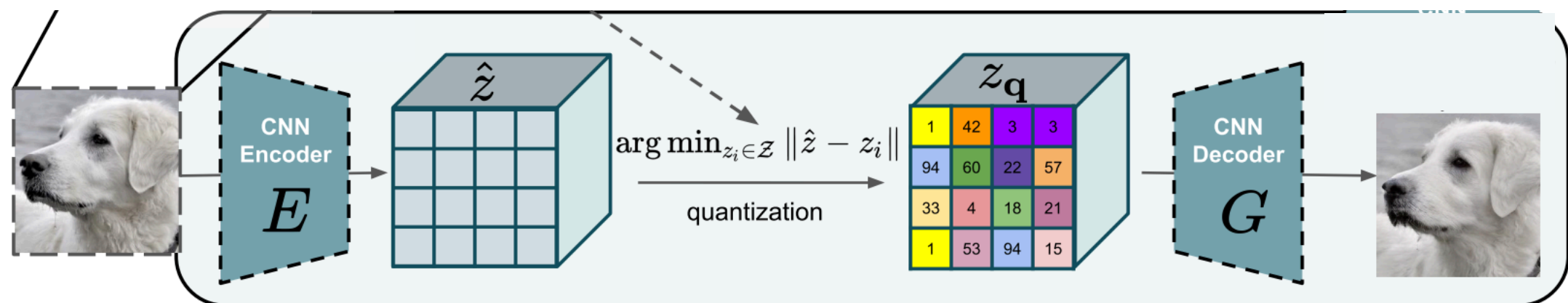
VQ-GAN

VQ-GAN

- Given the benefits of adversarial loss in GANs, can we “borrow” this functionality in other generative models?
- Yes!
 - VAE-GAN (e.g., Larsen et al. 2015)
 - VQ-GAN (e.g., Esser et al. 2021)

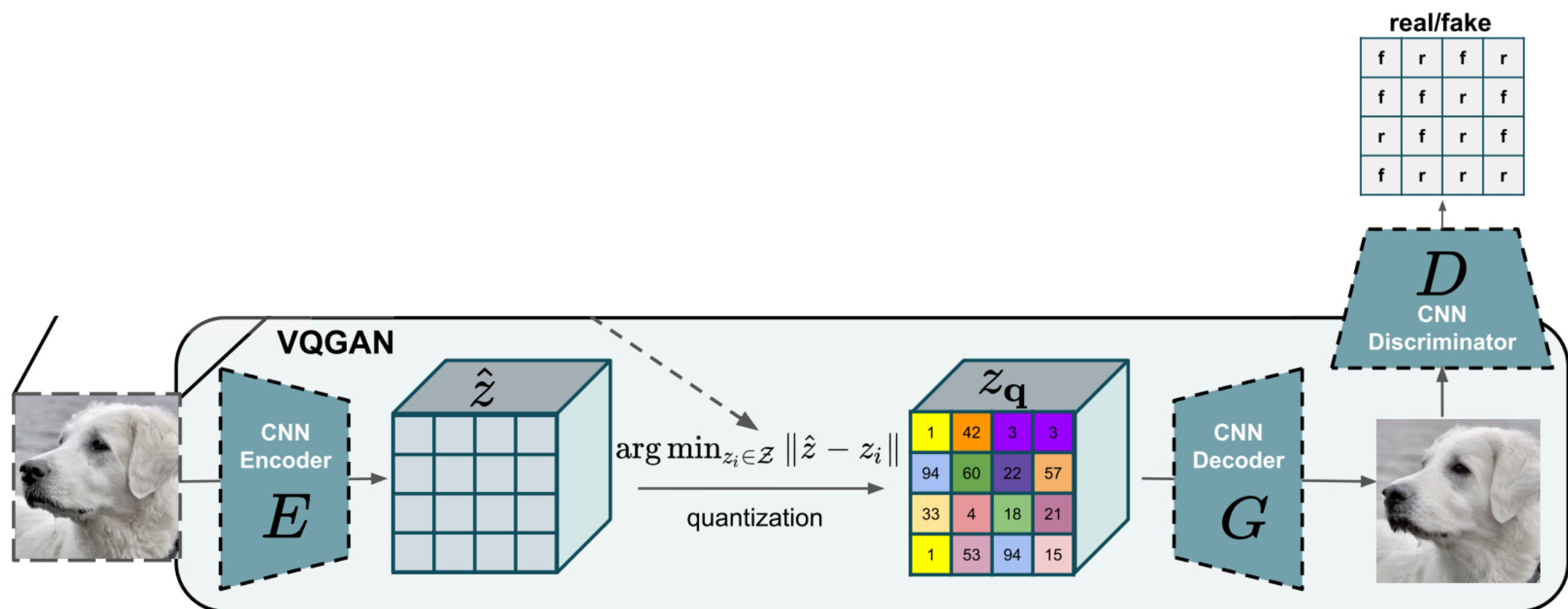
VQ-GAN (Esser et al. 2021)

- Instead of a regular VQ-VAE...



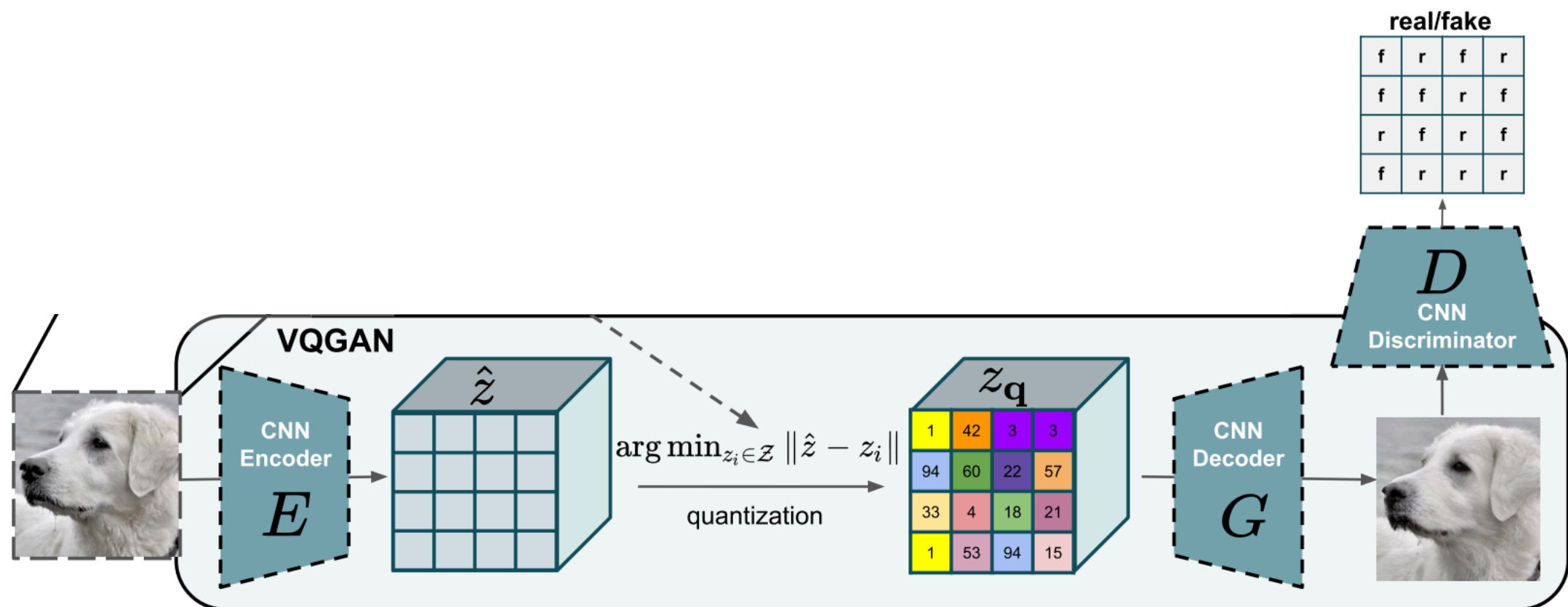
VQ-GAN (Esser et al. 2021)

- Instead of a regular VQ-VAE...we additionally apply an **adversarial loss** with a trained discriminator D on patch-wise outputs of the decoder G .



VQ-GAN (Esser et al. 2021)

- Instead of a regular VQ-VAE...we additionally apply an **adversarial loss** with a trained discriminator D on patch-wise outputs of the decoder G .
- This helps to remove blurriness and improve visual quality.



Exercise 1

[3 pts] A generative adversarial network (GAN) consists of a discriminator (with parameters θ) trained jointly with a generator (with parameters ϕ). What function does the *generator* try to *minimize* with respect to the generator's network parameters ϕ ? Select one answer below.

(a) $f(\theta, \phi) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta}(\mathbf{x})]$

(b) $f(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta}(G_{\phi}(\mathbf{z})))]$

(c) $f(\theta, \phi) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta}(\mathbf{x})] - \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta}(G_{\phi}(\mathbf{z})))]$

Exercise 2

[4 pts] Which of the following statements is/are true about the input vector $\mathbf{z}$ to the generator of a GAN? You may select any number of answers.

- (a) If G is trained successfully, then $\mathbf{z}$ is transformed by G to produce a novel example that resembles the training data.
- (b) The probability distribution $P(\mathbf{z})$ over the random vectors $\mathbf{z}$ is typically a complicated distribution that is difficult to sample from.
- (c) During training, $\mathbf{z}$ is a noise-augmented example from the training set.
- (d) $\mathbf{z}$ is sampled using the “re-parameterization trick”, whereby the random sampling (a non-differentiable operation) is brought outside of the computational graph.

Exercise 3

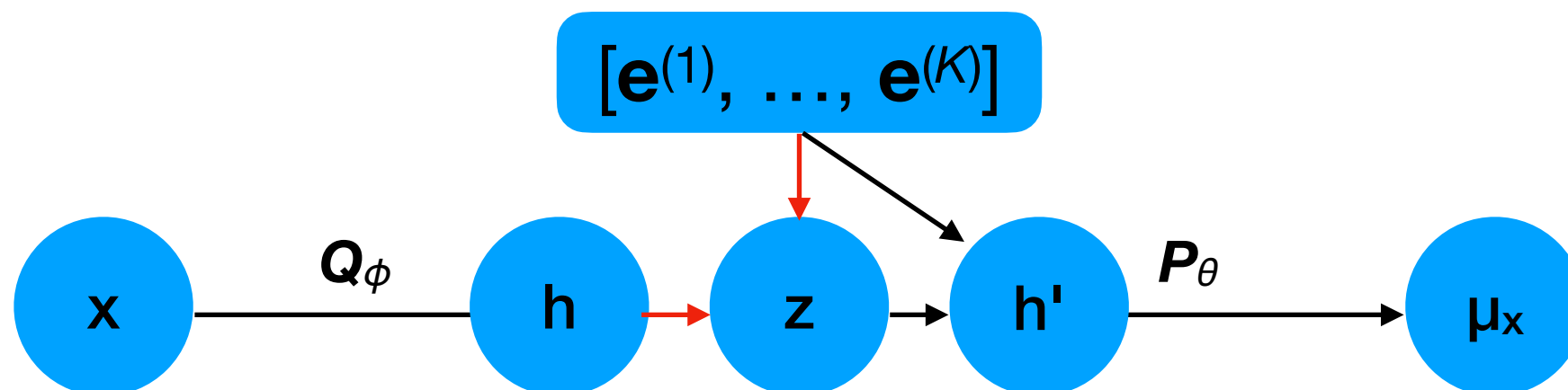
- Why might the generator suffer from vanishing gradients if the discriminator is too strong (and thus it is very confident in its predictions on fake data)?

Exercise 4

- Suppose a GAN is perfectly trained and thus reaches the equilibrium solution (i.e., neither D nor G can improve without making the other network worse). What is the output $D(\mathbf{x})$ on a fake example $\mathbf{x}$?

Exercise 5

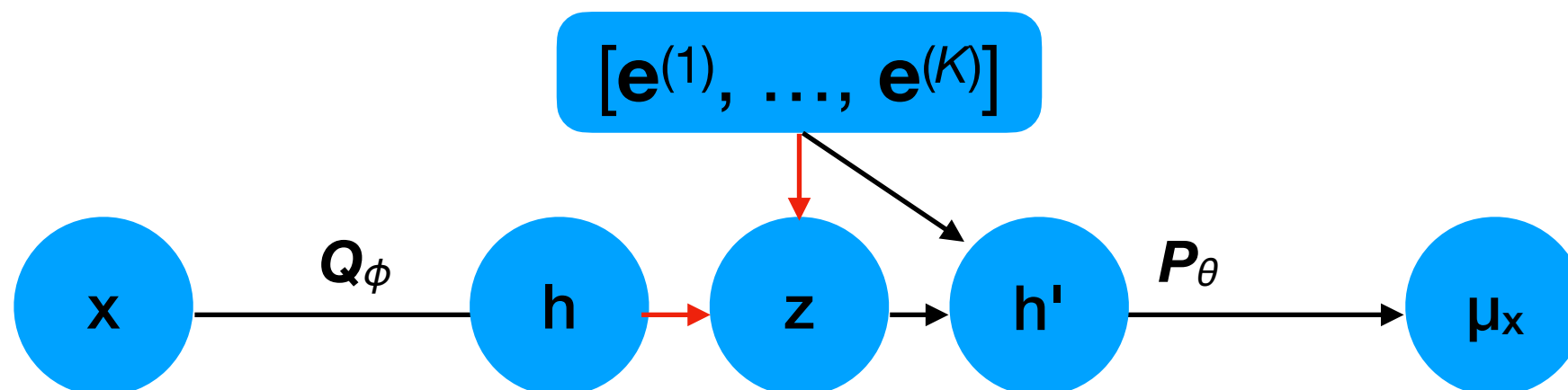
- For VQ-VAEs, draw a figure of the quantized vectors in the latent space to explain why the Jacobian $\frac{\partial \mathbf{h}'}{\partial \mathbf{h}}$ needed for training is either 0 or undefined.



Exercise 6

- Explain the role that each term of the VQ-VAE loss function plays during training:

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$

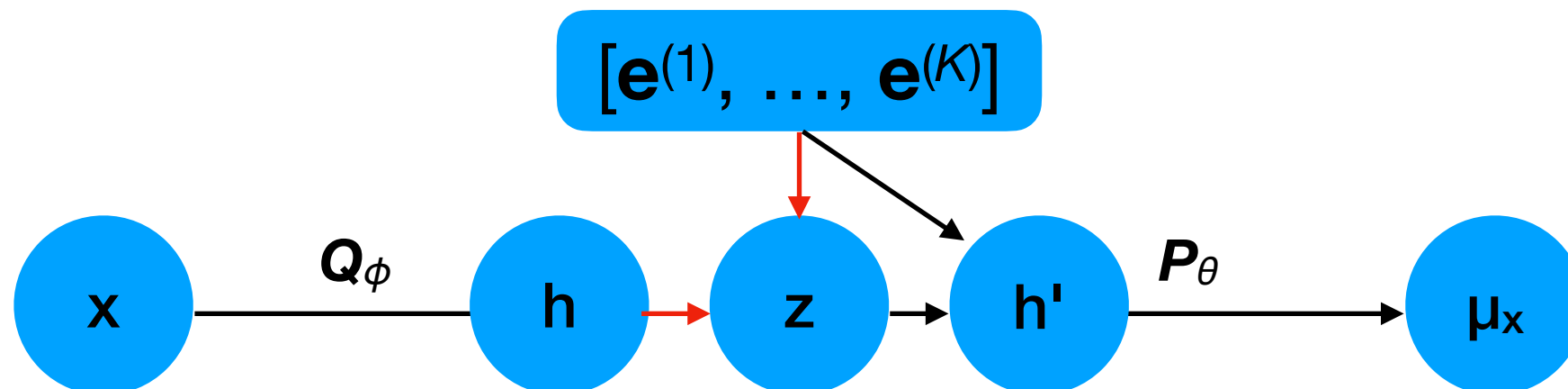


Exercise 7

- While in theory the ELBO derived for VQE applies to VQ-VAEs as well, in practice it is useless.

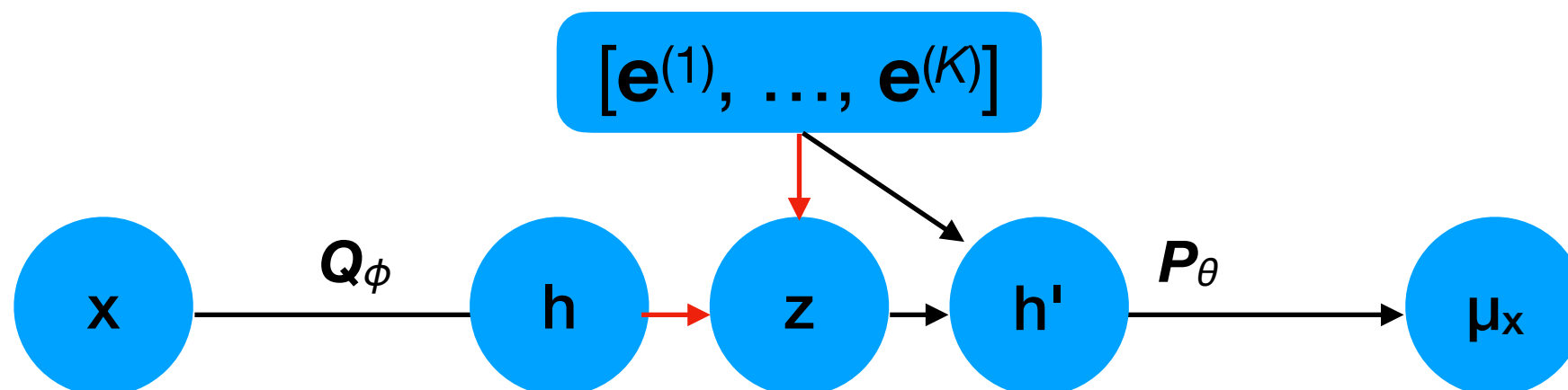
$$-D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$

- Explain why neither term is useful for training a VQ-VAE.



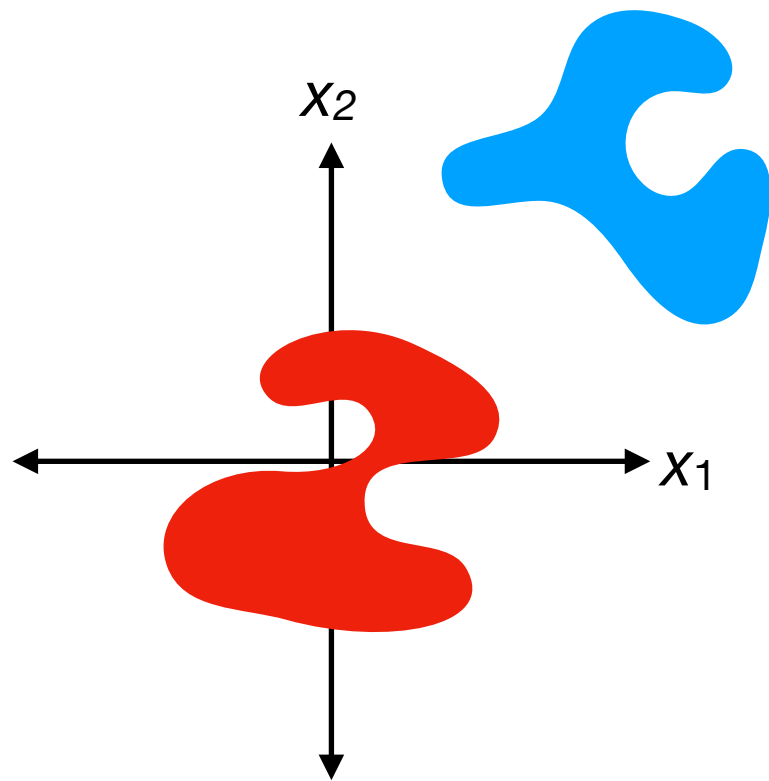
Exercise 8

- What is the practical implication of the lack of meaningful KL divergence term when using a trained VQ-VAE for generation?

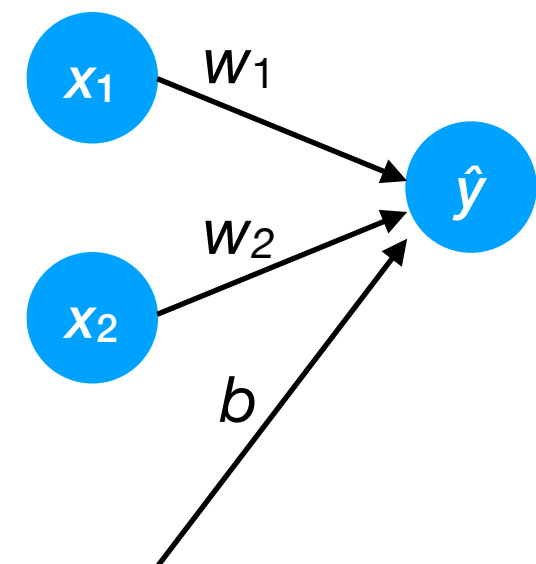


Exercise 9

- Suppose the red region depicts the probability distribution of generated data and the blue region depicts the true data distribution. Set the neural network weights and bias to act as the discriminator D (with sigmoid activation function).



Discriminator D



Exercise 10

- Suppose the red region depicts the probability distribution of generated data and the blue region depicts the true data distribution. How might a discriminator D distinguish the real from the fake data?

